

KUBERNETES (K8S) TASK - 03

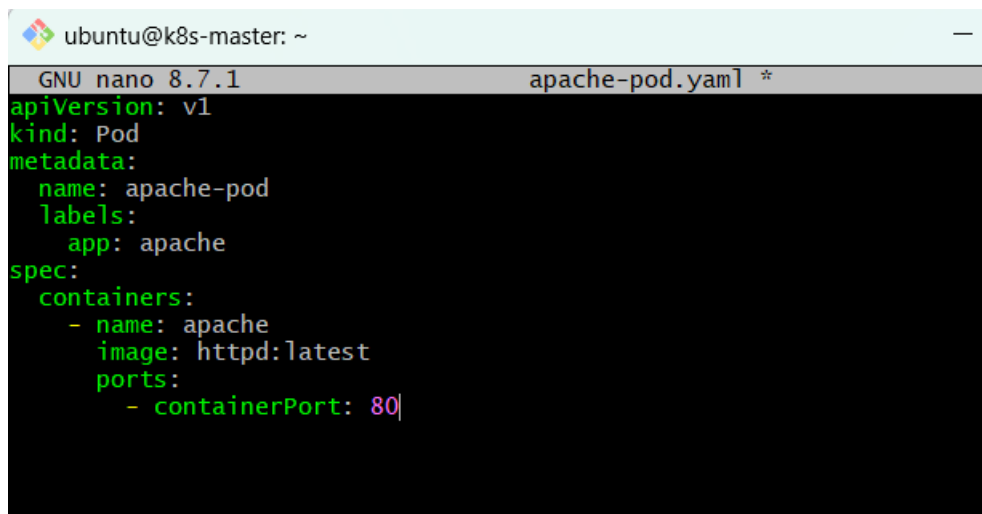
1. Create a ClusterIP service for an Apache web server pod.

Objective: Create an Apache pod and expose it internally using a ClusterIP Service.

Step 1: Create Apache Pod

Create the YAML file:

nano apache-pod.yaml



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 apache-pod.yaml *  
apiVersion: v1  
kind: Pod  
metadata:  
  name: apache-pod  
  labels:  
    app: apache  
spec:  
  containers:  
    - name: apache  
      image: httpd:latest  
      ports:  
        - containerPort: 80
```

Step 2: Create the Pod

kubectl apply -f apache-pod.yaml

Step 3: Verify

kubectl get pods

Expected:

NAME	READY	STATUS	RESTARTS	AGE
apache-pod	1/1	Running	0	...

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f apache-pod.yaml  
pod/apache-pod created  
ubuntu@k8s-master:~$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
apache-pod                          1/1     Running   0           4s  
edit-deployment-89c465f4f-rwvz7    1/1     Running   0           24s  
ubuntu@k8s-master:~$ |
```

Step 4: Create ClusterIP Service

nano apache-service.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 apache-service.yaml *  
apiVersion: v1  
kind: Service  
metadata:  
  name: apache-service  
spec:  
  type: ClusterIP  
  selector:  
    app: apache  
  ports:  
    - port: 80  
      targetPort: 80
```

Apply:

kubectl apply -f apache-service.yaml

Step 5: Verify Service

kubectl get svc

Expected:

NAME	TYPE	CLUSTER-IP	PORT(S)
apache-service	ClusterIP	10.x.x.x	80/TCP

Step 6: Verify Endpoints

kubectl get endpoints apache-service

You should see the Apache pod's IP:

NAME	ENDPOINTS
------	-----------

apache-service	10.244.x.x:80
----------------	---------------

Important

ClusterIP is **internal to the Kubernetes cluster**.

You normally cannot access it directly from your laptop/Internet.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f apache-service.yaml  
service/apache-service created  
ubuntu@k8s-master:~$ kubectl get svc  
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE  
apache-service       ClusterIP    10.98.222.38   <none>       80/TCP     26s  
firstpod-service     ClusterIP    10.98.115.180  <none>       80/TCP     21h  
kubernetes            ClusterIP    10.96.0.1      <none>       443/TCP    44h  
ubuntu@k8s-master:~$ kubectl get endpoints apache-service  
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 Endpointslice  
NAME                ENDPOINTS          AGE  
apache-service       192.168.194.79:80  43s  
ubuntu@k8s-master:~$ kubectl get endpoints apache-service  
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice  
NAME                ENDPOINTS          AGE  
apache-service       192.168.194.79:80  88s  
ubuntu@k8s-master:~$ |
```

2. Expose an Nginx pod externally using a NodePort service.

Objective: Create an Nginx pod and expose it using a **NodePort Service**.

Step 1: Create Nginx Pod

nano nginx-nodeport.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 nginx-nodeport.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx-nodeport-pod  
  labels:  
    app: nginx-nodeport  
spec:  
  containers:  
    - name: nginx  
      image: nginx:latest  
      ports:  
        - containerPort: 80|
```

Apply:

```
kubectl apply -f nginx-nodeport.yaml
```

Verify:

```
kubectl get pods -o wide
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano nginx-nodeport.yaml  
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport.yaml  
pod/nginx-nodeport-pod created  
ubuntu@k8s-master:~$ kubectl get pods -o wide  
NAME             NODE             NOMINATED NODE   READY   STATUS    RESTARTS   AGE   IP              
apache-pod       k8s-worker1      <none>           1/1     Running   0           7m22s  192.168.1.     
94.79            edit-deployment-89c465f4f-rwvz7 1/1             <none>   0           7m42s  192.168.1.     
26.15            k8s-worker2      <none>           1/1     Running   0           9s      192.168.1.     
nginx-nodeport-pod k8s-worker2      <none>           1/1     Running   0           18s     192.168.126.16  
ubuntu@k8s-master:~$ kubectl get pods -o wide  
NAME             READY   STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES  
apache-pod       1/1     Running   0           7m31s  192.168.194.79 k8s-worker1      <none>           <none>  
edit-deployment-89c465f4f-rwvz7 1/1     Running   0           7m51s  192.168.126.15 k8s-worker2      <none>           <none>  
nginx-nodeport-pod 1/1     Running   0           18s     192.168.126.16 k8s-worker2      <none>           <none>  
ubuntu@k8s-master:~$ |
```

Step 2: Create NodePort Service

```
nano nginx-nodeport-service.yaml
```

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 nginx-nodeport-se  
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-nodeport-service  
spec:  
  type: NodePort  
  selector:  
    app: nginx-nodeport  
  ports:  
    - port: 80  
      targetPort: 80  
      nodePort: 30080
```

Apply:

```
kubectl apply -f nginx-nodeport-service.yaml
```

Step 3: Verify

```
kubectl get svc nginx-nodeport-serviceHere:
```

80 = Service port

30080 = NodePort

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano nginx-nodeport-service.yaml  
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport-service.yaml  
service/nginx-nodeport-service created  
ubuntu@k8s-master:~$ kubectl get svc nginx-nodeport-service  
NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)  
nginx-nodeport-service             NodePort      10.103.57.154    <none>           80:30080/TCP  
7s  
ubuntu@k8s-master:~$ |
```

Step 4: Get Worker Node IP

`kubectl get nodes -o wide`

Find the **Internal IP** or use the EC2 public IP of a worker node.

From your browser:

`http://<WORKER_PUBLIC_IP>:30080`

You should see:

Welcome to nginx!

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes -o wide  
NAME                                STATUS    ROLES    AGE    VERSION    INTERNAL-IP    EXTERNAL-IP  
k8s-master                         Ready    control-plane  45h    v1.34.11    172.31.17.127    <none>  
k8s-worker1                         Ready    <none>    44h    v1.34.11    172.31.16.155    <none>  
k8s-worker2                         Ready    <none>    44h    v1.34.11    172.31.26.51    <none>  
ubuntu@k8s-master:~$ kubectl get nodes -o wide  
NAME                                STATUS    ROLES    AGE    VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE          KERNEL-VERSION    CONTAINER-RUNTIME  
k8s-master                         Ready    control-plane  45h    v1.34.11    172.31.17.127    <none>          Ubuntu 26.04.1 LTS  7.0.0-1012-aws    containerd://2.2.2  
k8s-worker1                         Ready    <none>    44h    v1.34.11    172.31.16.155    <none>          Ubuntu 26.04 LTS   7.0.0-1012-aws    containerd://2.2.2  
k8s-worker2                         Ready    <none>    44h    v1.34.11    172.31.26.51    <none>          Ubuntu 26.04 LTS   7.0.0-1012-aws    containerd://2.2.2  
ubuntu@k8s-master:~$ |
```

AWS Important Step

Because your Kubernetes nodes are AWS EC2 instances, make sure the EC2 Security Group allows:

Custom TCP

Port: 30080

Source: Your IP

For a temporary lab, you could use 0.0.0.0/0, but restricting it to your IP is safer.



3. Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.

Objective: Create a ReplicationController (RC) that maintains 3 Nginx pods.

ReplicationController is an older Kubernetes workload object. For new applications, ReplicaSet/Deployment is generally preferred, but RC is useful for learning.

Step 1: Create YAML

nano nginx-rc.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 nginx-rc.yaml  
apiVersion: v1  
kind: ReplicationController  
metadata:  
  name: nginx-rc  
spec:  
  replicas: 3  
  selector:  
    app: nginx-rc  
  template:  
    metadata:  
      labels:  
        app: nginx-rc  
    spec:  
      containers:  
        - name: nginx  
          image: nginx:latest  
          ports:  
            - containerPort: 80
```

Step 2: Create RC

kubectl apply -f nginx-rc.yaml

Step 3: Verify RC

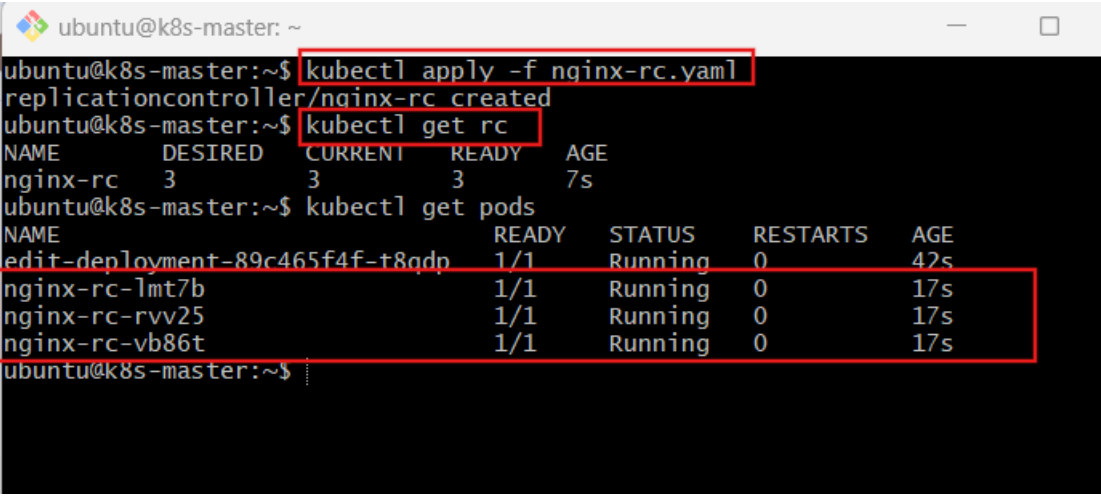
kubectl get rc

Expected:

NAME	DESIRED	CURRENT	READY
nginx-rc	3	3	3

Step 4: Check Pods

kubectl get pods



```
ubuntu@k8s-master: ~$ kubectl apply -f nginx-rc.yaml
replicationcontroller/nginx-rc created
ubuntu@k8s-master: ~$ kubectl get rc
NAME          DESIRED  CURRENT  READY  AGE
nginx-rc      3        3        3      7s
ubuntu@k8s-master: ~$ kubectl get pods
NAME                                READY  STATUS   RESTARTS  AGE
edit-deployment-89c465f4f-t8qdp    1/1    Running   0          42s
nginx-rc-1mt7b                      1/1    Running   0          17s
nginx-rc-rvv25                      1/1    Running   0          17s
nginx-rc-vb86t                      1/1    Running   0          17s
ubuntu@k8s-master: ~$
```

Step 5: Detailed Verification

kubectl describe rc nginx-rc

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl describe rc nginx-rc
Name:          nginx-rc
Namespace:     default
Selector:      app=nginx-rc
Labels:        app=nginx-rc
Annotations:   <none>
Replicas:      3 current / 3 desired
Pods Status:   3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx-rc
  Containers:
    nginx:
      Image:      nginx:latest
      Port:       80/TCP
      Host Port:  0/TCP
      Environment: <none>
      Mounts:      <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:   <none>
Events:
  Type     Reason                  Age    From                      Message
  ----     -
  Normal   SuccessfulCreate        118s   replication-controller    Created pod: nginx-rc-rvv25
  Normal   SuccessfulCreate        118s   replication-controller    Created pod: nginx-rc-vb86t
  Normal   SuccessfulCreate        118s   replication-controller    Created pod: nginx-rc-lmt7b
ubuntu@k8s-master:~$

```

4. Scale the ReplicationController from 3 replicas to 5 replicas.

Objective: Increase the number of Nginx pods from 3 → 5.

Step 1: Scale

```
kubectl scale rc nginx-rc --replicas=5
```

Step 2: Verify RC

```
kubectl get rc
```

Expected:

NAME	DESIRED	CURRENT	READY
nginx-rc	5	5	5

Step 3: Verify Pods

```
kubectl get pods -l app=nginx-rc
```

Expected:

NAME	READY	STATUS
nginx-rc-xxxxxx	1/1	Running
nginx-rc-yyyyyy	1/1	Running
nginx-rc-zzzzzz	1/1	Running
nginx-rc-aaaaa	1/1	Running
nginx-rc-bbbbbb	1/1	Running

You now have 5 Nginx replicas.

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl scale rc nginx-rc --replicas=5
replicationcontroller/nginx-rc scaled
ubuntu@k8s-master:~$ kubectl get rc
NAME          DESIRED  CURRENT  READY  AGE
nginx-rc      5        5        5      4m15s
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-rc
NAME          READY  STATUS   RESTARTS  AGE
nginx-rc-gnpwr 1/1    Running  0         19s
nginx-rc-lmt7b 1/1    Running  0         4m25s
nginx-rc-rvv25 1/1    Running  0         4m25s
nginx-rc-vb86t 1/1    Running  0         4m25s
nginx-rc-wst2l 1/1    Running  0         19s
ubuntu@k8s-master:~$ |

```

5. Create a ReplicaSet to manage pods based on multiple labels (prod and test).

Objective: Create pods with multiple labels such as:

app=web

environment=prod

and configure a ReplicaSet to select pods using both labels.

Step 1: Create ReplicaSet

nano nginx-rs-multiple-labels.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: ReplicaSet  
metadata:  
  name: nginx-rs-multiple-labels  
spec:  
  replicas: 3  
  
  selector:  
    matchLabels:  
      app: nginx  
      environment: prod  
  
  template:  
    metadata:  
      labels:  
        app: nginx  
        environment: prod  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:latest  
        ports:  
        - containerPort: 80
```

Step 2: Apply

kubectl apply -f nginx-rs-multiple-labels.yaml

Step 3: Verify

kubectl get rs

Step 4: Check Pods

kubectl get pods --show-labels

You should see:

nginx-rs-multiple-labels-xxxxx Running app=nginx,environment=prod

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get rs  
NAME                                DESIRED  CURRENT  READY  AGE  
edit-deployment-89c465f4f          1         1        1      22h  
edit-deployment-c8d57947           0         0        0      22h  
nginx-rs-multiple-labels           3         3        3      43s  
ubuntu@k8s-master:~$ kubectl get pods --show-labels  
NAME                                READY  STATUS   RESTARTS  AGE    LABELS  
edit-deployment-89c465f4f-t8qdp    1/1    Running  0          9m37s  app=edit-demo,pod-template-hash=89c465f4f  
nginx-rs-gnpwr                     1/1    Running  0          5m6s   app=nginx-rs  
nginx-rs-1mt7b                     1/1    Running  0          9m12s  app=nginx-rs  
nginx-rs-rvv25                     1/1    Running  0          9m12s  app=nginx-rs  
nginx-rs-vb86t                     1/1    Running  0          9m12s  app=nginx-rs  
nginx-rs-wst2l                     1/1    Running  0          5m6s   app=nginx-rs  
nginx-rs-multiple-labels-8s4v9     1/1    Running  0          48s    app=nginx,environment=prod  
nginx-rs-multiple-labels-b5smp     1/1    Running  0          48s    app=nginx,environment=prod  
nginx-rs-multiple-labels-pn16z     1/1    Running  0          48s    app=nginx,environment=prod  
ubuntu@k8s-master:~$
```

Step 5: Verify ReplicaSet Selector

kubectl describe rs nginx-rs-multiple-labels

Look for:

Selector: app=nginx,environment=prod

Understanding

The ReplicaSet requires **both labels**:

app=nginx

AND

environment=prod

This is an important concept with matchLabels.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl describe rs nginx-rs-multiple-labels  
Name:          nginx-rs-multiple-labels  
Namespace:     default  
Selector:      app=nginx,environment=prod  
Labels:        <none>  
Annotations:   <none>  
Replicas:      3 current / 3 desired  
Pods Status:   3 Running / 0 Waiting / 0 Succeeded / 0 Failed  
Pod Template:  
  Labels:      app=nginx  
              environment=prod  
Containers:  
  nginx:  
    Image:      nginx:latest  
    Port:       80/TCP  
    Host Port:  0/TCP  
    Environment: <none>  
    Mounts:     <none>  
  Volumes:     <none>  
  Node-Selectors: <none>  
  Tolerations:  <none>  
Events:  
  Type      Reason      Age   From          Message  
  ----      -  
  Normal    SuccessfulCreate 17m   replicaset-controller Created pod: nginx-rs-multiple-labels-pn16z  
  Normal    SuccessfulCreate 17m   replicaset-controller Created pod: nginx-rs-multiple-labels-b5smp  
  Normal    SuccessfulCreate 17m   replicaset-controller Created pod: nginx-rs-multiple-labels-8s4v9  
ubuntu@k8s-master:~$
```

6. Deploy a ReplicaSet that excludes pods with the label backend.

This task requires a little care because Kubernetes label selectors work primarily by **matching what should be selected**.

We can create a ReplicaSet that selects pods where:

app=web

environment != backend

Step 1: Create YAML

nano nginx-rs-exclude-backend.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: ReplicaSet  
metadata:  
  name: nginx-rs-exclude-backend  
spec:  
  replicas: 3  
  
  selector:  
    matchExpressions:  
      - key: app  
        operator: In  
        values:  
          - web  
  
      - key: environment  
        operator: NotIn  
        values:  
          - backend  
  
  template:  
    metadata:  
      labels:  
        app: web  
        environment: frontend  
    spec:  
      containers:  
        - name: nginx  
          image: nginx:latest  
          ports:  
            - containerPort: 80
```

Step 2: Apply

kubectl apply -f nginx-rs-exclude-backend.yaml

Step 3: Verify

kubectl get rs

Step 4: Check Pods and Labels

kubectl get pods --show-labels

The ReplicaSet-created pods will have:

app=web

environment=frontend

and not:

environment=backend

Important Concept

This:

operator: NotIn

values:

- backend

means:

environment must NOT be backend

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f nginx-rs-exclude-backend.yaml  
replicaset.apps/nginx-rs-exclude-backend created  
ubuntu@k8s-master:~$ kubectl get rs  
NAME                                DESIRED    CURRENT    READY    AGE  
edit-deployment-89c465f4f           1           1           1        22h  
edit-deployment-c8d57947            0           0           0        22h  
nginx-rs-exclude-backend            3           3           3         8s  
nginx-rs-multiple-labels             3           3           3        22m  
ubuntu@k8s-master:~$ kubectl get pods --show-labels  
NAME                                READY      STATUS    RESTARTS   AGE    LABELS  
edit-deployment-89c465f4f-t8qdp     1/1        Running   0           32m    app=edit-demo,pod-template-hash=89c465f4f  
nginx-rc-gnpwr                      1/1        Running   0           27m    app=nginx-rc  
nginx-rc-lmt7b                      1/1        Running   0           31m    app=nginx-rc  
nginx-rc-rvv25                      1/1        Running   0           31m    app=nginx-rc  
nginx-rc-vb86t                      1/1        Running   0           31m    app=nginx-rc  
nginx-rc-wst2l                      1/1        Running   0           27m    app=nginx-rc  
nginx-rs-exclude-backend-6vpm6       1/1        Running   0           20s    app=web,environment=frontend  
nginx-rs-exclude-backend-q2n68       1/1        Running   0           20s    app=web,environment=frontend  
nginx-rs-exclude-backend-zcjm8       1/1        Running   0           20s    app=web,environment=frontend  
nginx-rs-multiple-labels-8s4v9       1/1        Running   0           23m    app=nginx,environment=prod  
nginx-rs-multiple-labels-b5smp       1/1        Running   0           23m    app=nginx,environment=prod  
nginx-rs-multiple-labels-pn16z       1/1        Running   0           23m    app=nginx,environment=prod  
ubuntu@k8s-master:~$
```

7. Test load balancing across multiple pods using a NodePort service.

Objective: Create multiple Nginx pods and expose them through one NodePort Service.

Traffic sent to the Service can be distributed among the Service's backend pods.

We will use the 5-pod RC from Task 4.

Step 1: Create NodePort Service

nano nginx-loadbalance-service.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 nginx-loadbalance-service.yaml  
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-loadbalance  
spec:  
  type: NodePort  
  
  selector:  
    app: nginx-rc  
  
  ports:  
    - port: 80  
      targetPort: 80  
      nodePort: 30081
```

Apply:

```
kubectl apply -f nginx-loadbalance-service.yaml
```

Step 2: Verify Service

```
kubectl get svc nginx-loadbalance
```

Step 3: Verify Endpoints

```
kubectl get endpoints nginx-loadbalance
```

You should see multiple pod IP addresses:

```
nginx-loadbalance 10.244.x.x:80,10.244.x.x:80,10.244.x.x:80,...
```

Since you have 5 replicas, you should have approximately 5 endpoints.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano nginx-loadbalance-service.yaml  
ubuntu@k8s-master:~$ kubectl apply -f nginx-loadbalance-service.yaml  
service/nginx-loadbalance created  
ubuntu@k8s-master:~$ kubectl get svc nginx-loadbalance  
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE  
nginx-loadbalance    NodePort    10.103.212.154 <none>         80:30081/TCP     8s  
ubuntu@k8s-master:~$ kubectl get endpoints nginx-loadbalance  
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 Endpointslice  
NAME                ENDPOINTS  
nginx-loadbalance    192.168.126.24:80,192.168.126.25:80,192.168.126.26:80 + 2 more... 22s  
ubuntu@k8s-master:~$ kubectl get endpoints nginx-loadbalance  
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice  
NAME                ENDPOINTS  
nginx-loadbalance    192.168.126.24:80,192.168.126.25:80,192.168.126.26:80 + 2 more... 28s  
ubuntu@k8s-master:~$
```

Step 4: Find Worker IP

```
kubectl get nodes -o wide
```

```
ubuntu@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP
k8s-master    Ready     control-plane  45h   v1.34.11   172.31.17.127 <none>
k8s-worker1    Ready     <none>    45h   v1.34.11   172.31.16.155 <none>
k8s-worker2    Ready     <none>    45h   v1.34.11   172.31.26.51  <none>
ubuntu@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE           KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane  45h   v1.34.11   172.31.17.127 <none>          Ubuntu 26.04.1 LTS  7.0.0-1012-aws    containerd://2.2.2
k8s-worker1    Ready     <none>    45h   v1.34.11   172.31.16.155 <none>          Ubuntu 26.04 LTS   7.0.0-1012-aws    containerd://2.2.2
k8s-worker2    Ready     <none>    45h   v1.34.11   172.31.26.51  <none>          Ubuntu 26.04 LTS   7.0.0-1012-aws    containerd://2.2.2
ubuntu@k8s-master:~$
```

Use the appropriate EC2 public IP.

Open:

`http://<WORKER_PUBLIC_IP>:30081`



Step 5: Test Multiple Requests

From a machine that can reach the NodePort:

```
for i in {1..20}; do curl -s http://<WORKER_PUBLIC_IP>:30081 | grep -o "<title>.*</title>"; done
```

The Service forwards requests to its backend endpoints.

Better way to visibly demonstrate load balancing

Give each Nginx pod a different response.

First list the pods:

```
kubectl get pods -l app=nginx-rc -o wide
```

Then you can modify each pod's Nginx content individually for demonstration purposes.

For example:

```
kubectl exec <pod-name> -- sh -c 'echo "Response from POD-1" >
/usr/share/nginx/html/index.html'
```

Repeat for the other pods:

```
kubectl exec <pod-name> -- sh -c 'echo "Response from POD-2" >
/usr/share/nginx/html/index.html'
```

Then repeatedly access:

```
curl http://<WORKER_PUBLIC_IP>:30081
```

You can observe responses from different backend pods.

Note: Kubernetes Service load balancing distributes connections/traffic; repeated HTTP requests do not necessarily guarantee a neat 1→2→3→4→5 rotation.

```
ubuntu@k8s-master: ~  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
<title>Welcome to nginx!</title>  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-rc -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE  
NOMINATED NODE READINESS GATES  
nginx-rc-fmxcf 1/1     Running   0           10m   192.168.126.25 k8s-worker2  
  <none>      <none>  
nginx-rc-jtscv 1/1     Running   0           10m   192.168.194.88 k8s-worker1  
  <none>      <none>  
nginx-rc-sckq7 1/1     Running   0           10m   192.168.126.26 k8s-worker2  
  <none>      <none>  
nginx-rc-sm7vk 1/1     Running   0           10m   192.168.194.89 k8s-worker1  
  <none>      <none>  
nginx-rc-vws4m 1/1     Running   0           10m   192.168.126.24 k8s-worker2  
  <none>      <none>  
ubuntu@k8s-master:~$ kubectl exec <pod-name> -- sh -c 'echo "Response from POD-1"  
> /usr/share/nginx/html/index.html'  
-bash: pod-name: No such file or directory  
ubuntu@k8s-master:~$ kubectl exec <pod-name> -- sh -c 'echo "Response from POD-2"  
> /usr/share/nginx/html/index.html'  
-bash: pod-name: No such file or directory  
ubuntu@k8s-master:~$ kubectl get pods  
NAME          READY   STATUS    RESTARTS   AGE  
edit-deployment-89c465f4f-gwbtt 1/1     Running   0           11m  
nginx-rc-fmxcf 1/1     Running   0           11m  
nginx-rc-jtscv 1/1     Running   0           11m  
nginx-rc-sckq7 1/1     Running   0           11m  
nginx-rc-sm7vk 1/1     Running   0           11m  
nginx-rc-vws4m 1/1     Running   0           11m  
nginx-rs-exclude-backend-2xtb9 1/1     Running   0           11m  
nginx-rs-exclude-backend-6vqvz 1/1     Running   0           11m  
nginx-rs-exclude-backend-ffpw5 1/1     Running   0           11m  
nginx-rs-multiple-labels-bvwmq 1/1     Running   0           11m  
nginx-rs-multiple-labels-kglc2 1/1     Running   0           11m  
nginx-rs-multiple-labels-nq7bg 1/1     Running   0           11m  
ubuntu@k8s-master:~$ kubectl exec nginx-rc-fmxcf -- sh -c 'echo "Response from POD-1" > /usr/share/nginx/html/index.html'  
ubuntu@k8s-master:~$ curl http://98.81.141.151:30081  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to nginx!</title>  
<style>  
html { color-scheme: light dark; }  
body { width: 35em; margin: 0 auto;  
font-family: Tahoma, Verdana, Arial, sans-serif; }  
</style>  
</head>  
<body>
```

8. Delete a ReplicationController without affecting the running pods.

Objective: Delete the ReplicationController but **keep its existing pods running**.

Normally:

```
kubectl delete rc nginx-rc
```

would also delete the pods owned by the RC.

Therefore, use:

```
--cascade=orphan
```

Step 1: Check RC

```
kubectl get rc
```

You should see:

```
nginx-rc 5 5 5
```

Step 2: Check Pods

```
kubectl get pods -l app=nginx-rc
```

You should see 5 pods.

Step 3: Delete RC and Orphan Pods

```
kubectl delete rc nginx-rc --cascade=orphan
```

Expected:

```
replicationcontroller "nginx-rc" deleted
```

Step 4: Verify RC

```
kubectl get rc
```

nginx-rc should no longer exist.

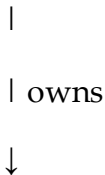
Step 5: Verify Pods

`kubectl get pods -l app=nginx-rc`

The 5 pods should still be running.

This happens because:

ReplicationController



Pod 1

Pod 2

Pod 3

Pod 4

Pod 5

With:

`--cascade=orphan`

Kubernetes removes the owner relationship but does **not** delete the pods.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get rc  
No resources found in default namespace.  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-rc  
No resources found in default namespace.  
ubuntu@k8s-master:~$ kubectl delete rc nginx-rc --cascade=orphan  
Error from server (NotFound): replicationcontrollers "nginx-rc" not found  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-rc  
No resources found in default namespace.  
ubuntu@k8s-master:~$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
edit-deployment-89c465f4f-gwbbt    1/1     Running   0           17m  
nginx-rs-exclude-backend-2xtb9      1/1     Running   0           17m  
nginx-rs-exclude-backend-6vqvb      1/1     Running   0           17m  
nginx-rs-exclude-backend-ffpw5      1/1     Running   0           17m  
nginx-rs-multiple-labels-bvwmq       1/1     Running   0           17m  
nginx-rs-multiple-labels-kglc2       1/1     Running   0           17m  
nginx-rs-multiple-labels-nq7bg       1/1     Running   0           17m  
ubuntu@k8s-master:~$
```

Complete Verification Commands

After completing the exercises, these commands are useful for your recording/documentation:

Nodes

```
kubectl get nodes
```

All Pods

```
kubectl get pods -o wide
```

Pods with labels

```
kubectl get pods --show-labels
```

Services

```
kubectl get svc
```

Endpoints

```
kubectl get endpoints
```

ReplicationControllers

```
kubectl get rc
```

ReplicaSets

```
kubectl get rs
```

Detailed Pod information

```
kubectl describe pod <pod-name>
```

Detailed Service information

```
kubectl describe svc <service-name>
```

Detailed ReplicaSet information

```
kubectl describe rs <replicaset-name>
```

Lab Summary

Task	Kubernetes Object	Main Command
1	ClusterIP Service	<code>kubectl apply -f apache-service.yaml</code>
2	NodePort Service	<code>kubectl apply -f nginx-nodeport-service.yaml</code>
3	ReplicationController	<code>kubectl apply -f nginx-rc.yaml</code>
4	Scale RC	<code>kubectl scale rc nginx-rc --replicas=5</code>
5	ReplicaSet + multiple labels	<code>kubectl apply -f nginx-rs-multiple-labels.yaml</code>
6	ReplicaSet excluding backend	<code>kubectl apply -f nginx-rs-exclude-backend.yaml</code>
7	NodePort load balancing	<code>kubectl apply -f nginx-loadbalance-service.yaml</code>
8	Delete RC, keep pods	<code>kubectl delete rc nginx-rc --cascade=orphan</code>

Key concepts to remember

ClusterIP



Internal cluster access

NodePort



External access through Node IP + NodePort

ReplicationController



Maintains desired number of Pods

ReplicaSet



Maintains desired number of Pods using label selectors

Service



Stable network endpoint



Routes traffic to matching Pods

--cascade=orphan



Delete controller



Keep existing Pods running

One important cleanup note: Tasks 3–8 create several resources with overlapping labels. If you run everything in the same namespace, keep an eye on selectors so one Service/ReplicaSet doesn't unintentionally match another task's pods. Using unique labels such as `app: nginx-rc`, `app: nginx-rs`, etc. avoids selector conflicts.